REPUBLIC OF TURKEY
YILDIZ TECHNICAL UNIVERSITY
DEPARTMENT OF COMPUTER ENGINEERING

CLASSIFICATION OF HYPERSPECTRAL IMAGES USING MACHINE LEARNING AND DEEP LEARNING METHODS

22011083 — TAN ERCİYAS
22011084 — KEREM BAYDAR

SENIOR PROJECT

Advisor
Dr. Ahmet ELBİR

June, 2026

ACKNOWLEDGEMENTS

We are deeply grateful to our advisor, Dr. Ahmet ELBİR, whose guidance, patience, and technical insight shaped this project at every stage. His willingness to engage with both the theoretical and practical dimensions of our work made a genuine difference.

TAN ERCİYAS
KEREM BAYDAR

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

1D-CNN	1-Dimensional Convolutional Neural Network
3D-CNN	3-Dimensional Convolutional Neural Network
AE	Autoencoder
R&D	Research and Development
CNN	Convolutional Neural Networks
GAN	Generative Adversarial Networks
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HSI	Hyperspectral Images
MRF	Markov Random Fields
PCA	Principal Component Analysis
RF	Random Forest
RNN	Recurrent Neural Networks
SAE	Stacked Autoencoders
SVM	Support Vector Machines
ViT	Vision Transformer

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

CLASSIFICATION OF HYPERSPECTRAL IMAGES USING MACHINE LEARNING AND DEEP LEARNING METHODS

TAN ERCİYAS
KEREM BAYDAR

Department of Computer Engineering
Senior Project

Advisor: Dr. Ahmet ELBİR

This project develops a unified classification framework for hyperspectral images (HSI) that spans supervised, semi-supervised, and federated learning paradigms, with the overarching goal of characterizing how each approach handles the dual challenge of high spectral dimensionality and scarce labeled samples in remote sensing contexts. Three deep learning architectures form the core of the system: 2D-CNN, 3D-CNN, and Vision Transformer (ViT). Raw spectral data exceeding 200 bands is reduced to 30 principal components through PCA, preceded by Gaussian filtering for noise suppression and channel-wise normalization. Semi-supervised training is realized via a self-training strategy augmented by curriculum learning, which structures the presentation of unlabeled samples from confident to uncertain predictions to stabilize optimization. Privacy-preserving collaborative training is achieved through a simulated server-client federated architecture operating over three standard benchmark datasets: Indian Pines, Pavia University, and Salinas. All models are assessed on Overall Accuracy, Precision, Recall, F1-Score, and Cohen's Kappa coefficient. The results reveal a clear performance split. Semi-supervised learning reaches competitive accuracy on the two larger datasets (86–97%) but falls substantially short on the smaller Indian Pines set (54–74%), yielding an overall average below supervised baselines. Federated learning proves more resilient across datasets, incurring an average accuracy drop of only 3.1% relative to centralized training while preserving data privacy. A PyQt6-based GUI with multithreaded inference makes the trained models accessible for real-time HSI classification without

requiring code-level interaction.

Keywords: Hyperspectral Image Classification, Deep Learning, CNN, Vision Transformer, Semi-Supervised Learning, Federated Learning, Machine Learning, Curriculum Learning, Remote Sensing, Principal Component Analysis

ÖZET

HİPERSPEKTRAL GÖRÜNTÜLERİN MAKİNE ÖĞRENMESİ VE DERİN ÖĞRENME YÖNTEMLERİYLE SINIFLANDIRILMASI

TAN ERCİYAS
KEREM BAYDAR

Bilgisayar Mühendisliği Bölümü
Bitirme Projesi

Danışman: Dr. Ahmet ELBİR

Bu proje, hiperspektral görüntülerin (HSI) sınıflandırılması için denetimli, yarı-denetimli ve federe öğrenme paradigmalarını bir arada ele alan bütünleşik bir çerçeve sunmaktadır. Temel motivasyon, uzaktan algılama alanında sıklıkla karşılaşılan iki temel güçlükten kaynaklanmaktadır: yüksek spektral boyutsallık ve yetersiz etiketli örnek sayısı. Sistemin omurgasını üç derin öğrenme mimarisi oluşturmaktadır: 2D-CNN, 3D-CNN ve Vision Transformer (ViT). 200'ü aşkın spektral bantten oluşan ham veri, kanal bazlı normalizasyon ve Gaussian gürültü filtrelemesinin ardından Temel Bileşen Analizi (PCA) ile 30 bileşene indirgenmektedir. Yarı-denetimli eğitimde, müfredat öğrenmesiyle güçlendirilmiş bir kendinden eğitim stratejisi benimsenmiştir; bu yaklaşım, etiketsiz örnekleri güven skoruna göre kolaydan zora sıralayarak optimizasyon sürecini kararlı kılmaktadır. Federe öğrenme ise üç standart kıyaslama veri seti (Indian Pines, Pavia University ve Salinas) üzerinde çalışan simüle edilmiş bir sunucu-istemci mimarisi aracılığıyla hayata geçirilmiş, gizlilik korumalı dağıtık eğitime olanak tanımıştır. Modeller; Genel Doğruluk, Kesinlik, Geri Çağırma, F1-Puanı ve Cohen's Kappa katsayısı metrikleriyle karşılaştırmalı biçimde değerlendirilmiştir. Sonuçlar belirgin bir ayrışma ortaya koymaktadır. Yarı-denetimli öğrenme, iki büyük veri setinde rekabetçi doğruluk elde ederken (%86–97), daha küçük Indian Pines veri setinde denetimli temel çizgisinin önemli ölçüde gerisinde kalmaktadır (%54–74). Federe öğrenme ise veri setleri genelinde daha tutarlı bir performans sergileyerek merkezi eğitime kıyasla

yalnızca ortalama %3,1 doğruluk kaybıyla gizlilik güvencesi sunmaktadır. Geliştirilen PyQt6 tabanlı, çok iş parçacıklı GUI, eğitilmiş modelleri kod düzeyinde müdahale gerektirmeksizin gerçek zamanlı HSI sınıflandırması için erişilebilir kılmaktadır.

Anahtar Kelimeler: Hiperspektral Görüntü Sınıflandırması, Derin Öğrenme, CNN, Vision Transformer, Yarı-Denetimli Öğrenme, Federe Öğrenme, Makine Öğrenmesi, Müfredat Öğrenmesi, Uzaktan Algılama, Temel Bileşen Analizi

1
INTRODUCTION

Hyperspectral images (HSI) capture electromagnetic reflectance across hundreds of narrow spectral bands per pixel, encoding the physical and chemical properties of Earth's surface materials at a level of detail that conventional RGB imagery simply cannot achieve. This spectral richness makes HSI indispensable in agriculture, environmental monitoring, and defense applications. Yet the same abundance of information that gives HSI its analytical power also introduces a fundamental computational burden: the well-documented curse of dimensionality inflates training costs for machine learning and deep learning models, while the scarcity of expert-labeled pixel samples compounds the problem by severely limiting model generalization.

This project investigates the performance gap between supervised and semi-supervised deep learning architectures—CNN, ViT, and Self-Training—under these twin constraints of high dimensionality and labeled data scarcity. Two contributions distinguish it from prior work. First, Curriculum Learning is integrated into the training pipeline to stabilize optimization across all model variants. Second, a multithreaded Graphical User Interface (GUI) is developed so that users can load external data and obtain predictions in real time, without touching a single line of code.

The report is structured as follows. Chapter 2 surveys the relevant literature and situates the project within the current state of the art. Chapter 3 details the system design and methodology, covering data preprocessing through model architectures. Chapter 4 presents comparative experimental results across multiple performance metrics, and Chapter 5 discusses findings and outlines directions for future work.

1.1 Purpose

The primary objective of this project is to quantitatively characterize the performance differences between supervised and semi-supervised learning in the classification of high-dimensional hyperspectral data. This overarching goal breaks down into the following sub-objectives:

- To benchmark model performance on three standard hyperspectral datasets widely adopted in the literature: Indian Pines, Pavia University, and Salinas.
- To assess how preprocessing steps—Gaussian filtering for noise suppression and Principal Component Analysis (PCA) for dimensionality reduction—affect classification accuracy.
- To incorporate Curriculum Learning into each model to improve training stability and convergence behavior.
- To compare models objectively using Accuracy, Precision, Recall, F1-Score, and Kappa metrics.
- To deliver a Python-based multithreaded GUI that exposes the trained models to end users, enabling real-time prediction without requiring direct code interaction.

1.2 Preliminary Investigation

Surveying the hyperspectral classification literature reveals a clear trend: traditional machine learning methods are steadily being displaced by deep learning approaches. A recurring limitation across existing systems, however, is their dependence on substantial quantities of labeled data—a resource that is both costly and slow to produce, since pixel annotation requires domain expertise and substantial manual effort.

This project targets that gap directly. Semi-supervised algorithms are designed to exploit a small labeled set alongside a large pool of unlabeled data, and the experiments here test whether that design advantage translates into competitive classification performance under realistic labeling budgets. A second gap is more practical in nature: most research models exist only as scripts, inaccessible to non-technical stakeholders. The multithreaded GUI introduced in this project bridges that divide, turning experimental code into an application that accepts user-supplied data and returns instantaneous predictions.

2
LITERATURE ANALYSIS

Hyperspectral image (HSI) classification has occupied a central place in remote sensing research for decades, driven by the promise of resolving surface materials with spectral fidelity that broadband sensors cannot match. The trajectory of the field—from hand-crafted statistical classifiers to attention-based deep architectures—reflects a persistent tension between model expressiveness and the practical scarcity of labeled training data. This chapter surveys that trajectory, grouping methods by their algorithmic foundations and highlighting the gaps that motivate the present work.

2.1 Traditional Machine Learning Approaches

Early HSI classification relied heavily on statistical and shallow machine learning methods, with Support Vector Machines (SVM) and Random Forest (RF) emerging as the dominant frameworks. Du et al. leveraged discriminative sparse representations combined with multinomial logistic regression to reach 97.71% accuracy on the Indian Pines dataset [1]. Xia et al. pursued a complementary direction, applying extended morphological profiles and ensemble strategies to push RF performance further [2]. To counter overfitting in ensemble settings, cascaded RF architectures were introduced, training multiple classifiers in a sequential hierarchy [3]. SVM variants also proliferated: probabilistic SVMs paired with guided filters [4] and spectral-spatial formulations grounded in Markov Random Fields (MRF) [5] both demonstrated strong results on standard benchmarks.

These methods share notable virtues—noise robustness, interpretability, and modest computational requirements. Their ceiling, however, is structural. High-dimensional spectral spaces expose classical feature extraction to the curse of dimensionality, and hand-crafted spatial features consistently lag behind the learned representations that deep networks produce.

2.2 Convolutional Neural Network-Based Approaches

CNNs reshaped the HSI classification landscape by learning spatial and spectral features jointly, without manual feature engineering. Chen et al. applied 3D-CNN with L2 regularization and dropout, reaching 99.66% accuracy on the Pavia University dataset—a result that demonstrated 3D convolution's natural fit for volumetric spectral-spatial data [6]. Li et al. took a different tack, proposing a 1D-CNN that generates deep pixel-pair features to compensate for limited labeled samples [7].

The parameter cost of pure 3D-CNN models quickly became a bottleneck. Several architectures addressed this directly. HybridSN concatenates 3D and 2D convolutional stages to balance spectral-spatial expressiveness against computational burden [8]. Ahmad et al. designed fast, compact 3D-CNN variants that retain accuracy while substantially cutting inference time [9]. Paoletti et al. tackled boundary artifacts through border mirroring, reducing information loss at patch edges [10]. Collectively, these networks validate CNN-based feature learning for HSI classification—yet their appetite for labeled data and GPU memory grows proportionally with depth, a constraint that limits deployment in data-scarce real-world scenarios.

2.3 Autoencoders, Generative Networks, and RNNs

The labeled data bottleneck prompted exploration of architectures that can extract useful representations without full supervision. Stacked Autoencoders (SAE) emerged as a natural choice for unsupervised dimensionality reduction. Li et al. combined SAE with CNNs to learn locally adaptive spatial weights [11], while Zhao et al. integrated SAE-based compression with 3D deep residual networks for joint dimensionality reduction and classification [12].

Recurrent Neural Networks brought sequence-modeling capabilities to HSI. Shi et al. adapted RNNs to fuse multi-scale spectral-spatial features, treating spectral bands as a temporal sequence [13]. Generative Adversarial Networks offered a different remedy: synthetic sample generation to augment sparse labeled sets, directly targeting the generalization deficit caused by scarce annotations [14].

Each of these families carries practical drawbacks. SAE and RNN pipelines suffer from sequential processing overhead that limits scalability, and GAN training is notoriously unstable—small hyperparameter shifts can collapse the generator or produce mode-dropped distributions that corrupt the augmented training set.

2.4 Transformer Architectures and Semi-Supervised Methods

Transformers entered HSI classification as a mechanism for capturing long-range spectral-spatial dependencies that local convolution windows cannot reach. Hybrid architectures coupling CNN local feature extractors with Transformer global attention heads showed strong empirical results [15, 16]. Tokenization schemes that encode spectral-spatial patches as input sequences enabled more direct application of standard Vision Transformer designs [17], and dual attention networks—attending separately to spectral and spatial dimensions—pushed benchmark accuracy above 99% on several datasets [18].

That performance, impressive as it is, comes at a hardware cost that dwarfs any preceding approach. Transformers demand large training sets and substantial GPU resources, making them impractical in low-data or edge-deployment settings.

On the semi-supervised front, Sellami et al. demonstrated that 3D deep networks can exploit unlabeled data effectively when paired with adaptive band selection, narrowing the accuracy gap between labeled and unlabeled regimes [19]. The study was an important proof of concept. Its key open problems—decision boundary bias from pseudo-label noise and sensitivity to the choice of confidence threshold—remain largely unsolved in the literature.

2.5 Current Standing and Original Contribution of This Study

A consistent pattern runs through the reviewed literature: models that achieve the highest accuracy under abundant labeled data are precisely the models most vulnerable when labels are scarce, and vice versa. Transformer and deep CNN architectures overfit on small labeled sets; semi-supervised and generative approaches mitigate the data problem but introduce training instability and threshold sensitivity. No prior work integrates a stability mechanism directly into the semi-supervised training loop.

This project addresses that gap by unifying CNN and Transformer architectures under a Self-Training framework augmented with Curriculum Learning—a strategy that orders training samples by estimated difficulty to prevent the model from fixating on noisy pseudo-labels early in training. Beyond algorithmic novelty, most research models in this domain exist exclusively as command-line scripts, inaccessible to practitioners without deep learning expertise. The multithreaded PyQt6 GUI introduced here converts validated models into a user-facing application that accepts external data and returns predictions in real time, bridging the gap between academic benchmarks

and practical deployment.

3
SYSTEM ANALYSIS AND FEASIBILITY

System analysis serves as the bridge between problem identification and design: it defines what the system must do, establishes the constraints it must operate within, and sets the criteria against which its eventual output will be judged. This chapter details the project objectives, traces how requirements were elicited, and determines that the proposed solution is viable from technical, legal, economic, and workforce perspectives.

3.1 System Analysis

Because this is an R&D project, requirements were not handed down from a client specification—they emerged from a systematic examination of the performance bottlenecks that plague existing deep learning-based HSI classifiers. The system's core function is to ingest high-dimensional spectral data—either loaded externally by the user or drawn from established benchmark datasets (Indian Pines [20], Pavia University [21], and Salinas [22])—and assign a class label to each pixel with high accuracy.

Three functional requirements govern the system:

- Preprocessing raw hyperspectral cubes via Gaussian filtering for noise suppression and Principal Component Analysis (PCA) for dimensionality reduction [23].
- Training supervised (CNN [6], ViT [24]) and semi-supervised (Self-Training [25]) models under a Curriculum Learning schedule [26] that controls sample difficulty progression.
- Exposing all trained models through a multithreaded GUI that enables simultaneous model testing and real-time per-pixel prediction without requiring code-level interaction.

Satisfaction of these requirements will be measured using Accuracy, Precision, Recall, F1-Score, and Cohen's Kappa coefficient.

3.2 Feasibility

The feasibility study assesses the viability of the proposed approach across four dimensions: technical, legal, economic, and workforce.

3.2.1 Technical Feasibility

Python [27] was selected as the implementation language for its mature machine learning ecosystem and broad community support. Deep learning components are built on PyTorch [28] or TensorFlow [29]; data processing relies on Scikit-learn [23] and OpenCV [30]. The GUI is implemented in PyQt6, chosen specifically for its native support for multithreaded worker patterns—critical for keeping the interface responsive during long inference runs. Development takes place in VS Code [31] with version control managed through GitHub [32].

3.2.1.1 Hardware Feasibility

Training deep models on full hyperspectral cubes demands substantial parallel compute. GPU resources are supplied through Google Colab [33], eliminating the need for dedicated on-premises hardware. GUI development and local integration work are handled on the project team's existing macOS machines, which are more than adequate for the task.

3.2.2 Legal Feasibility

Every library in the stack—Python, PyQt6, PyTorch, OpenCV—is distributed under an open-source license. The three benchmark datasets are publicly available for academic use. No licensing conflicts or patent encumbrances have been identified.

3.2.3 Economic Feasibility

The entire toolchain is free and open source, and Google Colab covers the GPU compute cost at no charge. Combined with the team's existing personal hardware, the project incurs zero additional capital expenditure and is fully economically viable.

3.2.4 Workforce and Time Feasibility

The project is carried out by Tan Erciyas and Kerem Baydar. Between them, the team covers all technical domains required for delivery. The Gantt chart below maps the project timeline from February 2026 through June 2026.

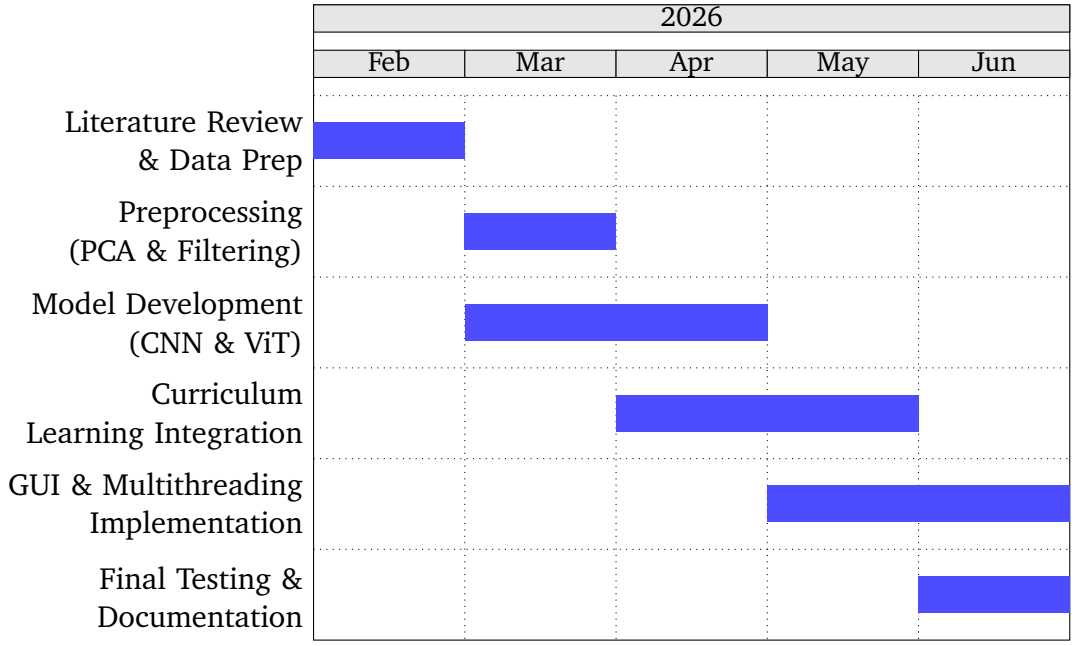

Figure 3.1 Project Gantt Chart (Feb 2026 - June 2026)

4
SYSTEM DESIGN

System design translates the requirements identified in the analysis phase into a concrete architectural framework. This chapter covers the datasets used, the preprocessing pipeline that conditions raw spectral data for learning, the three deep learning architectures at the core of the system, and the supervised, semi-supervised, and federated training paradigms evaluated against one another.

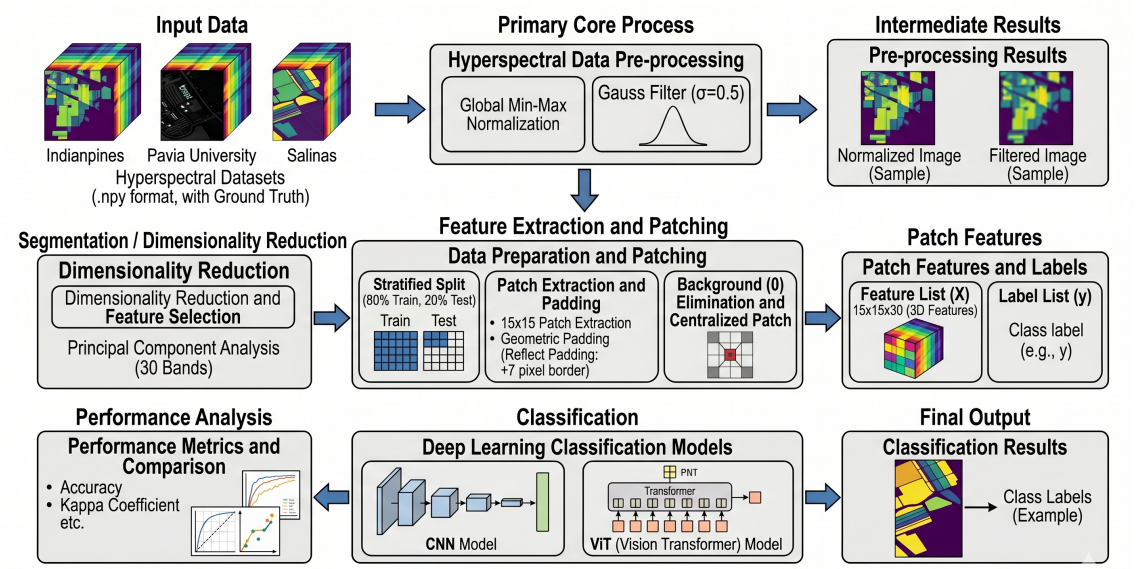

Figure 4.1 Overall Workflow of the Proposed HSI Classification System

4.1 Material and Dataset

Model performance in HSI classification is tightly coupled to dataset characteristics—spatial resolution, class imbalance, and total labeled sample count all shape what a given architecture can learn. Three benchmark datasets from the remote sensing literature are used here, each posing a distinct set of challenges.

4.1.1 Dataset Specifications

4.1.1.1 Indian Pines

Captured by the AVIRIS sensor over north-western Indiana, this dataset spans a 145 × 145 pixel grid across 224 spectral reflectance bands and covers 16 vegetation and land-cover classes. Water absorption bands reduce the usable band count to 200 in standard practice [20].

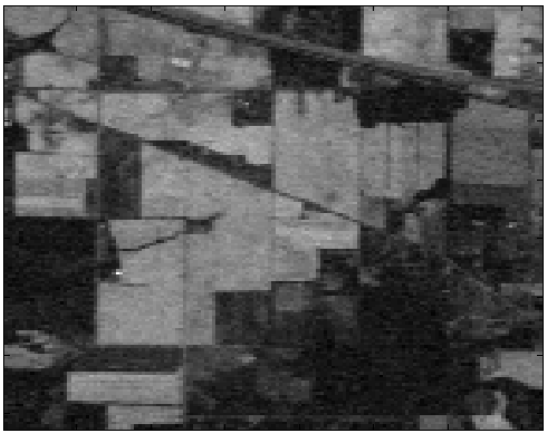

Figure 4.2 Indian Pines – False-color band composite

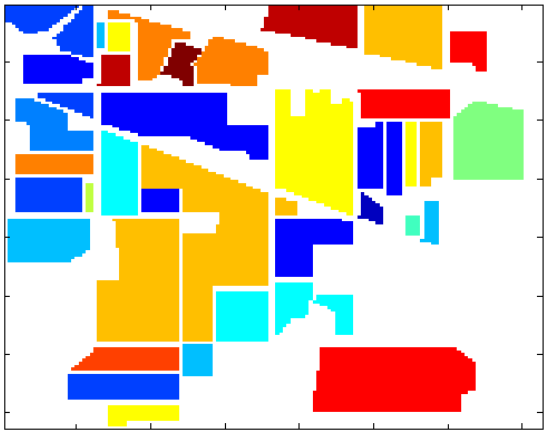

Figure 4.3 Indian Pines – Ground truth map

Table 4.1 Groundtruth classes for the Indian Pines scene and their respective samples number

#	**Class**	**Samples**
1	Alfalfa	46
2	Corn-notill	1428
3	Corn-mintill	830
4	Corn	237
5	Grass-pasture	483
6	Grass-trees	730
7	Grass-pasture-mowed	28
8	Hay-windrowed	478
9	Oats	20
10	Soybean-notill	972
11	Soybean-mintill	2455
12	Soybean-clean	593
13	Wheat	205
14	Woods	1265
15	Buildings-Grass-Trees-Drives	386
16	Stone-Steel-Towers	93
	Total	**10249**

4.1.1.2 Pavia University

Acquired by the ROSIS sensor over an urban area in Pavia, Italy, this scene covers 610×340 pixels across 103 spectral bands. Nine urban land-cover classes—including asphalt, meadows, and self-blocking bricks—make it a demanding multi-class urban benchmark [21].

Figure 4.4 Pavia University – False-color band composite

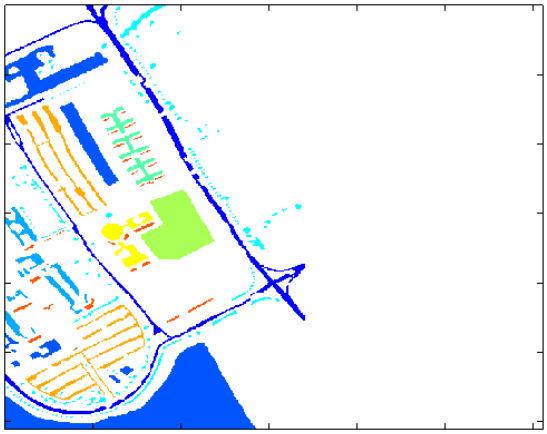

Figure 4.5 Pavia University – Ground truth map

Table 4.2 Groundtruth classes for the Pavia University scene and their respective samples number

#	**Class**	**Samples**
1	Asphalt	6631
2	Meadows	18649
3	Gravel	2099
4	Trees	3064
5	Painted metal sheets	1345
6	Bare Soil	5029
7	Bitumen	1330
8	Self-Blocking Bricks	3682
9	Shadows	947
	Total	**42776**

4.1.1.3 Salinas Scene

Also collected by AVIRIS, this time over Salinas Valley, California, the scene offers the highest spatial resolution of the three datasets: 512×217 pixels across 224 spectral bands, reduced to 204 after removing water absorption channels [22].

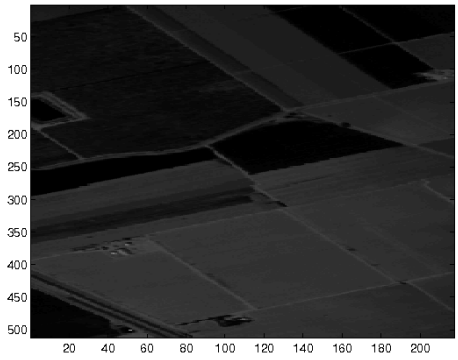

Figure 4.6 Salinas Scene – False-color band composite

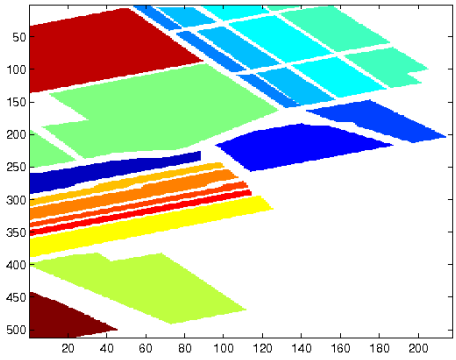

Figure 4.7 Salinas Scene – Ground truth map

Table 4.3 Groundtruth classes for the Salinas scene and their respective samples number

#	**Class**	**Samples**
1	Brocoli_green_weeds_1	2009
2	Brocoli_green_weeds_2	3726
3	Fallow	1976
4	Fallow_rough_plow	1394
5	Fallow_smooth	2678
6	Stubble	3959
7	Celery	3579
8	Grapes_untrained	11271
9	Soil_vinyard_develop	6203
10	Corn_senesced_green_weeds	3278
11	Lettuce_romaine_4wk	1068
12	Lettuce_romaine_5wk	1927
13	Lettuce_romaine_6wk	916
14	Lettuce_romaine_7wk	1070
15	Vinyard_untrained	7268
16	Vinyard_vertical_trellis	1807
	Total	**54129**

4.1.2 Data Structure and Storage

All three datasets are represented as 3D hypercubes of shape $H \times W \times B$, where H and W are the spatial dimensions and B is the number of spectral bands. Datasets are stored as NumPy arrays ('.npy') and loaded in spatial patches during training, which preserves local spatial context while keeping memory consumption tractable.

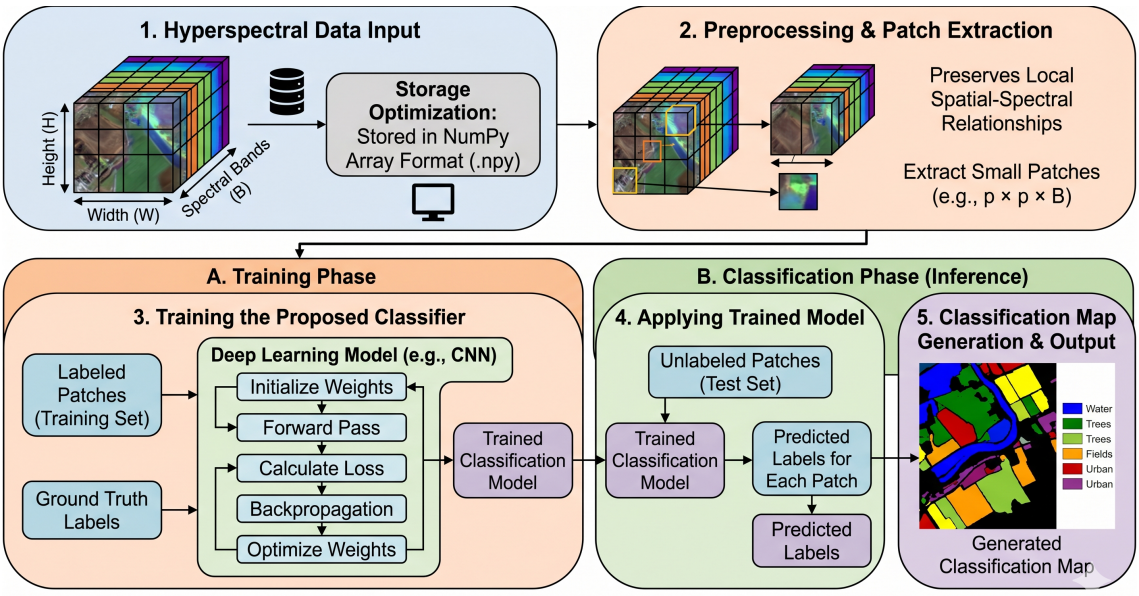

Figure 4.8 General Block Diagram of the Proposed HSI Classification System

4.2 Methodology

The processing pipeline is structured into three sequential stages: preprocessing, patch-based feature extraction, and the learning phase. Each stage feeds directly into the next, and the design choices at each step are motivated by specific properties of hyperspectral data.

4.2.1 Preprocessing and Dimensionality Reduction

Raw hyperspectral cubes carry hundreds of correlated bands and sensor-induced noise—conditions that inflate training cost and can mislead learned features. Three sequential operations are applied uniformly across all datasets before any model sees the data.

1. **Global Min-Max Normalization:** The raw hypercube is reshaped to a 2D matrix and a single global minimum and maximum is computed across all pixels and

all bands. Each value is then rescaled to $[0, 1]$ as follows:

$$\hat{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{4.1}$$

Band-wise normalization was deliberately avoided to preserve the natural variance ratio between bands.

2. **Spatial Gaussian Filtering:** A Gaussian filter is applied with $\sigma = 0.5$ along only the spatial axes ($H \times W$) of the normalized hypercube using `scipy.ndimage.gaussian_filter` with `sigma=(0.5, 0.5, 0)` and `mode='reflect'`. Setting the spectral axis sigma to zero ensures that inter-band relationships remain intact while sensor-induced spatial noise is attenuated.

3. **Fixed-Component PCA:** PCA with a fixed $n\_components = 30$ is applied to the filtered data after reshaping it to $(H \times W, B)$. The variance-based threshold (99.5%) was evaluated first but yielded only 4–5 components for Pavia University and Salinas, which is insufficient for the CNN and ViT input depths. The resulting reduced cube is reshaped back to $(H, W, 30)$ [23].

4.2.2 Patch Extraction and Dataset Splitting

For each labeled pixel (r, c) where `gt[r,c] > 0`, a 15×15 spatial patch is extracted from the PCA-reduced cube, producing a sample of shape $15 \times 15 \times 30$. Background pixels (label $= 0$) are excluded entirely. To allow boundary pixels to serve as valid patch centers, the cube is first padded by $\lfloor 15/2 \rfloor = 7$ pixels on each spatial side using reflect padding.

The resulting patches are split into training and test sets with `test_size=0.1`, `stratify=y`, and `random_state=42`, ensuring minority classes are proportionally represented in both splits. The resulting sample counts are given in Table 4.4.

Table 4.4 Patch counts after stratified 90/10 train-test split ($15 \times 15 \times 30$)

Dataset	**Total**	**Train (90%)**	**Test (10%)**
Indian Pines	10249	9224	1025
Pavia University	42776	38498	4278
Salinas Scene	54129	48716	5413

4.2.3 Model Architectures

Two architectures represent complementary inductive biases for spectral-spatial feature learning:

- **3D-CNN:** Three-dimensional convolutional kernels extract spectral and spatial features simultaneously from volumetric patches. Batch normalization and ReLU activations are inserted after each convolutional block to stabilize gradients and prevent vanishing activations [6].
- **Vision Transformer (ViT):** HSI patches are flattened into sequences of linear embeddings, and multi-head self-attention weighs each spectral-spatial token against every other—capturing long-range dependencies that local convolution windows cannot reach [24].

4.2.4 Supervised Learning Models

The supervised paradigm trains exclusively on the labeled portion of the training set. It establishes the performance baseline against which semi-supervised and federated variants are measured.

4.2.4.1 2D-CNN Supervised Model

The 2D-CNN operates on $15 \times 15 \times 30$ patches, treating the 30 PCA components as input channels and applying 2D convolution kernels over the spatial dimensions. This formulation avoids the parameter cost of full volumetric convolution while retaining meaningful spatial-spectral structure.

4.2.4.2 3D-CNN Supervised Model

The 3D-CNN applies volumetric kernels that span both spatial and spectral axes simultaneously, enabling it to model inter-band correlations directly rather than treating spectral bands as independent channels. Batch normalization and dropout mitigate overfitting, which is particularly pronounced when labeled samples are scarce.

4.2.4.3 Vision Transformer (ViT) Supervised Model

The ViT tokenizes each patch into a sequence of linear projections and augments them with positional encodings to retain spatial ordering. Multi-head self-attention then

relates every token to every other, giving the model a global receptive field from the first layer. This property has proven competitive on standard HSI benchmarks [24].

4.2.5 Semi-Supervised Learning Models

Collecting expert-labeled pixel annotations is expensive and slow. Semi-supervised learning sidesteps this bottleneck by treating the large pool of unlabeled pixels as additional training signal. The strategy here pairs self-training with curriculum learning to expand the labeled set iteratively while managing pseudo-label noise.

4.2.5.1 Self-Training Strategy

The self-training mechanism operates through iterative pseudo-labeling:

1. Initialize the model using a subset of labeled training samples.
2. Generate predictions on the unlabeled pool with associated confidence scores.
3. Incorporate samples exceeding a confidence threshold into the training set with their predicted labels.
4. Retrain the model on the augmented labeled set.
5. Repeat until convergence or a maximum iteration limit is reached [25].

4.2.5.2 Curriculum Learning Integration

Curriculum Learning (CL) guides the training process by presenting samples in order of increasing difficulty:

1. **Easy Samples:** Pixels with high spectral purity and clear spatial context are introduced first, allowing the model to learn robust decision boundaries.
2. **Medium Samples:** Pixels with moderate spectral-spatial characteristics are gradually introduced as the model gains confidence.
3. **Difficult Samples:** Boundary pixels and spectrally ambiguous samples are presented in later epochs to refine the decision boundaries [26].

4.2.5.3 2D-CNN Semi-Supervised Model

The architecture is identical to its supervised counterpart. What changes is the training set: self-training iteratively adds high-confidence pseudo-labeled patches, and curriculum learning controls the order in which those additions are presented, dampening the effect of early-stage label noise.

4.2.5.4 3D-CNN Semi-Supervised Model

Unlabeled samples admitted through self-training expose the 3D-CNN to a wider range of spectral-spatial patterns than the labeled set alone provides. Curriculum scheduling prevents the volumetric kernels from overfitting to the noisiest pseudo-labels encountered in early iterations.

4.2.5.5 Vision Transformer (ViT) Semi-Supervised Model

The global receptive field of the self-attention mechanism makes the ViT naturally suited to semi-supervised settings: it can relate high-confidence pseudo-labeled tokens to labeled ones across the full spatial extent of a patch, rather than relying solely on local neighborhood agreement.

4.2.6 Federated Learning Models

Federated learning trains a shared global model without ever moving raw data to a central server—each client trains locally and contributes only weight updates. In this project, federated learning is simulated through a server-client architecture in which three geographically distinct datasets stand in for independent client institutions [34].

4.2.6.1 Federated Learning Framework

The federated learning process follows the Federated Averaging (FedAvg) algorithm:

1. Initialize a global model at the central server.
2. Each client downloads the global model weights and trains locally on its own hyperspectral dataset for a fixed number of epochs.
3. Clients upload their updated model weights to the server (not raw data, preserving privacy).
4. The server aggregates weights from all participating clients through averaging.

5. The updated global model is redistributed to clients for the next round.
6. Repeat for multiple communication rounds until convergence.

4.2.6.2 2D-CNN Federated Model

Each client (Indian Pines, Pavia University, and Salinas in this simulation) trains an identical 2D-CNN locally under shared hyperparameters. Weight averaging at the server fuses the dataset-specific spatial patterns learned at each site into a single global model without exposing any client's raw pixels.

4.2.6.3 3D-CNN Federated Model

The 3D-CNN's volumetric kernels encode dataset-specific spectral correlations during local training. After aggregation, the global model inherits a richer spectral vocabulary than any single-dataset model could develop—particularly valuable in remote sensing, where data is routinely distributed across multiple acquisition campaigns and sensor configurations.

4.2.6.4 Vision Transformer (ViT) Federated Model

Transformer attention weights encode the local data distribution of each client site. Their aggregation at the server produces a global self-attention mechanism capable of handling the heterogeneous spectral-spatial statistics that arise when different sensors observe different geographic regions.

4.2.6.5 Privacy and Communication Efficiency

Keeping raw hyperspectral data at client locations is the primary privacy guarantee of federated learning, but communication overhead between clients and the server is a practical constraint. Three measures address this:

- **Gradient Compression:** Model weights are compressed before transmission to reduce bandwidth requirements.
- **Local Training Epochs:** Each client performs multiple local training epochs (typically 5–10) before communicating with the server, reducing the total number of communication rounds.
- **Asynchronous Aggregation:** The server aggregates updates from whichever clients respond first rather than waiting for all participants, reducing wall-clock training time.

4.2.7 Comparative Learning Paradigm Summary

The three paradigms each address labeled data scarcity from a different angle. Table 4.5 summarizes their key characteristics.

Table 4.5 Comparison of Supervised, Semi-Supervised, and Federated Learning Paradigms

Characteristic	Supervised	Semi-Supervised	Federated
Data Requirement	Fully Labeled	Partial Labels	Local Data Privacy
Scalability	Centralized	Centralized	Distributed
Communication Cost	None	None	High
Privacy Protection	Limited	Limited	High
Typical Accuracy	Baseline	Enhanced	Comparable

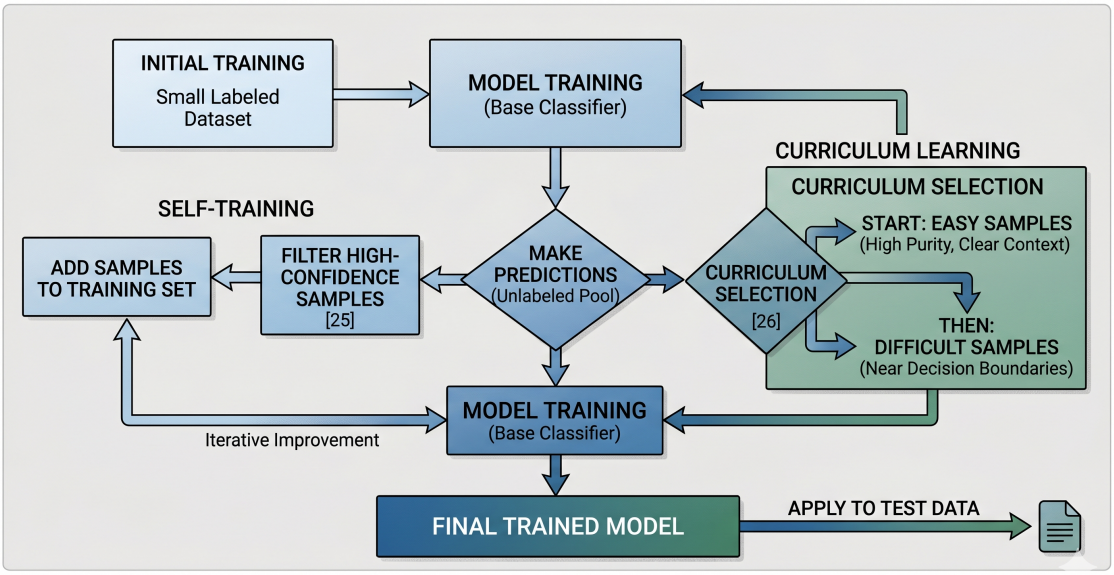

Figure 4.9 Flowchart of the Semi-Supervised Learning Strategy

4.2.8 Evaluation Metrics

Five standard metrics quantify classification performance. Let TP_c, FP_c, and FN_c denote the true positives, false positives, and false negatives for class c, and let N be the total number of test samples.

- **Overall Accuracy (OA):** The ratio of correctly classified pixels to the total number of test pixels:

$$\text{OA} = \frac{\sum_c TP_c}{N} \tag{4.2}$$

- **Precision:** The fraction of pixels predicted as class c that truly belong to class c, averaged across all classes (macro):

$$\text{Precision} = \frac{1}{C}\sum_{c=1}^{C}\frac{TP_c}{TP_c + FP_c} \tag{4.3}$$

- **Recall:** The fraction of true class-c pixels that are correctly identified:

$$\text{Recall} = \frac{1}{C}\sum_{c=1}^{C}\frac{TP_c}{TP_c + FN_c} \tag{4.4}$$

- **F1-Score:** The harmonic mean of Precision and Recall:

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4.5}$$

- **Cohen's Kappa (κ):** A metric that corrects Overall Accuracy for agreement expected by chance. Given the observed accuracy p_o and the chance agreement p_e:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{4.6}$$

  A value of $\kappa = 1$ indicates perfect classification; $\kappa = 0$ is equivalent to random guessing.

4.2.9 GUI and Multithreading Integration

The trained models are exposed through a standalone PyQt6 desktop application. Model inference runs in a dedicated worker thread, decoupled from the main UI event loop, so the interface remains fully responsive while the deep learning pipeline processes large hyperspectral hypercubes in the background.

5
EXPERIMENTAL RESULTS

5.1 Performance Analysis

All three architectures—2D-CNN, 3D-CNN, and Vision Transformer—are evaluated on Indian Pines, Pavia University, and Salinas under each of the three learning paradigms. The tables below report quantitative results; the comparative analysis that follows dissects the patterns across datasets and architectures.

5.1.1 Supervised Learning Results

Supervised models are trained exclusively on labeled samples, with hyperparameters selected through grid search and cross-validation. These results serve as the performance ceiling against which semi-supervised and federated variants are judged.

5.1.1.1 Indian Pines Dataset Performance

Table 5.1 Supervised Learning Performance on Indian Pines Dataset

Model	Overall Accuracy	Precision	Recall	F1-Score
2D-CNN	99.91%	0.9984	0.9987	0.9985
3D-CNN	99.90%	0.9977	0.9978	0.9976
ViT	99.61%	0.9959	0.9952	0.9953

5.1.1.2 Pavia University Dataset Performance

Table 5.2 Supervised Learning Performance on Pavia University Dataset

Model	Overall Accuracy	Precision	Recall	F1-Score
2D-CNN	99.99%	0.9998	0.9998	0.9998
3D-CNN	99.99%	0.9998	0.9998	0.9998
ViT	99.75%	0.9973	0.9961	0.9967

5.1.1.3 Salinas Dataset Performance

Table 5.3 Supervised Learning Performance on Salinas Dataset

Model	Overall Accuracy	Precision	Recall	F1-Score
2D-CNN	99.99%	0.9999	0.9999	0.9999
3D-CNN	99.84%	0.9993	0.9995	0.9994
ViT	99.81%	0.9979	0.9983	0.9981

5.1.2 Semi-Supervised Learning Results

The semi-supervised variants augment the labeled training set through iterative self-training with curriculum learning, leveraging the unlabeled pixel pool to improve generalization. The results are notably dataset-dependent: performance on the two larger datasets remains competitive, while the small, class-imbalanced Indian Pines set presents a harder challenge.

5.1.2.1 Semi-Supervised Indian Pines Performance

Table 5.4 Semi-Supervised Learning Performance on Indian Pines Dataset

Model	Overall Accuracy	Precision	Recall	F1-Score
2D-CNN + SSL	74.04%	0.7404	0.7404	0.7404
3D-CNN + SSL	66.55%	0.6655	0.6655	0.6655
ViT + SSL	53.96%	0.5396	0.5396	0.5396

5.1.2.2 Semi-Supervised Pavia University Performance

Table 5.5 Semi-Supervised Learning Performance on Pavia University Dataset

Model	Overall Accuracy	Precision	Recall	F1-Score
2D-CNN + SSL	97.17%	0.9717	0.9717	0.9717
3D-CNN + SSL	96.91%	0.9691	0.9691	0.9691
ViT + SSL	91.77%	0.9177	0.9177	0.9177

5.1.2.3 Semi-Supervised Salinas Performance

Table 5.6 Semi-Supervised Learning Performance on Salinas Dataset

Model	Overall Accuracy	Precision	Recall	F1-Score
2D-CNN + SSL	96.46%	0.9646	0.9646	0.9646
3D-CNN + SSL	97.30%	0.9730	0.9730	0.9730
ViT + SSL	86.49%	0.8649	0.8649	0.8649

5.1.3 Federated Learning Results

In the federated setting, each dataset acts as an independent client. Local models are trained and their weights aggregated via FedAvg over 50 communication rounds—raw data never leaves the client. The results show that federated learning sustains near-supervised accuracy on the two larger datasets while incurring a more meaningful gap on the small Indian Pines set.

5.1.3.1 Federated Learning on Indian Pines

Table 5.7 Federated Learning Performance on Indian Pines Dataset (Round 50)

Model	Overall Accuracy	Precision	Recall	F1-Score
2D-CNN (FedAvg)	96.43%	0.9156	0.8272	0.8466
3D-CNN (FedAvg)	92.56%	0.7603	0.7054	0.7019
ViT (FedAvg)	85.65%	0.7852	0.7257	0.7452

5.1.3.2 Federated Learning on Pavia University

Table 5.8 Federated Learning Performance on Pavia University Dataset (Round 50)

Model	Overall Accuracy	Precision	Recall	F1-Score
2D-CNN (FedAvg)	99.83%	0.9979	0.9967	0.9973
3D-CNN (FedAvg)	99.76%	0.9951	0.9970	0.9960
ViT (FedAvg)	98.95%	0.9887	0.9762	0.9819

5.1.3.3 Federated Learning on Salinas

Table 5.9 Federated Learning Performance on Salinas Dataset (Round 50)

Model	Overall Accuracy	Precision	Recall	F1-Score
2D-CNN (FedAvg)	99.90%	0.9989	0.9988	0.9988
3D-CNN (FedAvg)	99.76%	0.9979	0.9987	0.9983
ViT (FedAvg)	97.73%	0.9882	0.9899	0.9888

5.2 Comparative Analysis

With all results in hand, three dimensions of comparison are examined: the accuracy change each paradigm produces relative to the supervised baseline, performance differences across architectures, and dataset-specific trends.

5.2.1 Overall Accuracy Comparison

Table 5.10 Overall Accuracy Comparison Across All Learning Paradigms

Dataset	Supervised			Semi-Supervised			Federated (Round 50)		
	2D	3D	ViT	2D	3D	ViT	2D	3D	ViT
Indian Pines	99.91%	99.90%	99.61%	74.04%	66.55%	53.96%	96.43%	92.56%	85.65%
Pavia University	99.99%	99.99%	99.75%	97.17%	96.91%	91.77%	99.83%	99.76%	98.95%
Salinas	99.99%	99.84%	99.81%	96.46%	97.30%	86.49%	99.90%	99.76%	97.73%

5.2.2 F1-Score Comparison

Table 5.11 F1-Score Comparison Across All Learning Paradigms

Dataset	Supervised			Semi-Supervised			Federated (Round 50)		
	2D	3D	ViT	2D	3D	ViT	2D	3D	ViT
Indian Pines	0.9985	0.9976	0.9953	0.7404	0.6655	0.5396	0.8466	0.7019	0.7452
Pavia University	0.9998	0.9998	0.9967	0.9717	0.9691	0.9177	0.9973	0.9960	0.9819
Salinas	0.9999	0.9994	0.9981	0.9646	0.9730	0.8649	0.9988	0.9983	0.9888

5.2.3 Performance Change Analysis

Accuracy change relative to the supervised baseline is quantified as:

$$\text{Change\%} = \frac{\text{OA}_{\text{alt}} - \text{OA}_{\text{Supervised}}}{\text{OA}_{\text{Supervised}}} \times 100 \tag{5.1}$$

5.2.3.1 Semi-Supervised vs. Supervised

Semi-supervised learning degrades substantially relative to the supervised baseline, with the scale of that degradation determined almost entirely by dataset size:

- **Indian Pines:** Average drop of −34.96 percentage points across architectures (from 99.81% to 64.85%), indicating that the self-training strategy is highly sensitive to limited dataset size and class imbalance.
- **Pavia University:** Average drop of −4.63 percentage points (from 99.91% to 95.28%), demonstrating that curriculum learning partially mitigates degradation when larger unlabeled pools are available.
- **Salinas:** Average drop of −6.46 percentage points (from 99.88% to 93.42%), suggesting that even the largest dataset is insufficient to recover fully supervised-level accuracy through pseudo-labeling alone.

5.2.3.2 Federated vs. Supervised

Federated learning holds much closer to the supervised ceiling, particularly on the larger datasets:

- **Indian Pines:** Average difference of −8.26 percentage points (from 99.81% to 91.55%), reflecting the challenge of federated aggregation on a small dataset with limited local samples per client.

- **Pavia University:** Average difference of −0.40 percentage points (from 99.91% to 99.51%), demonstrating strong robustness across heterogeneous urban land-cover classes.
- **Salinas:** Average difference of −0.75 percentage points (from 99.88% to 99.13%), showing near-parity performance at scale.

5.2.4 Architecture Comparison

Table 5.12 Average Performance Across Datasets by Architecture Type

Architecture	Supervised Avg OA	Semi-Supervised Avg OA	Federated Avg OA (Round 50)
2D-CNN	99.96%	89.22%	98.72%
3D-CNN	99.91%	86.92%	97.36%
Vision Transformer	99.72%	77.41%	94.11%

Table 5.13 Average F1-Score Across Datasets by Architecture Type

Architecture	Supervised Avg F1	Semi-Supervised Avg F1	Federated Avg F1 (Round 50)
2D-CNN	0.9994	0.8922	0.9476
3D-CNN	0.9989	0.8692	0.8987
Vision Transformer	0.9967	0.7741	0.9053

Three patterns stand out from the architecture-level averages:

1. **2D-CNN dominates all paradigms:** Despite being the simplest architecture, 2D-CNN leads in supervised (99.96%), semi-supervised (89.22%), and federated (98.72%) average accuracy. After PCA reduces the spectral dimension to 30 components, 2D spatial convolutions appear fully sufficient to discriminate between classes—volumetric operations add complexity without a commensurate accuracy gain.
2. **3D-CNN offers consistent robustness:** The 3D-CNN ranks second across all three paradigms (supervised: 99.91%, semi-supervised: 86.92%, federated: 97.36%). Its explicit modeling of inter-band correlations through volumetric kernels contributes to stable performance even when training conditions are imperfect.
3. **ViT is most sensitive to distribution shift:** The Vision Transformer reaches competitive supervised accuracy (99.72%) but suffers the steepest

degradation under semi-supervised (77.41%) and federated (94.11%) settings. Self-attention mechanisms appear more susceptible to noisy pseudo-labels and to weight averaging across heterogeneous client data distributions.

5.2.5 Dataset-Specific Analysis

- **Indian Pines:** With only 10,249 samples spread across 16 imbalanced classes, this is the hardest setting for both semi-supervised and federated training. Semi-supervised accuracy collapses by an average of 34.96 percentage points (99.81% → 64.85%), driven by error propagation through pseudo-label iterations on severely underrepresented minority classes.
- **Pavia University:** At 42,776 samples across 9 urban classes, this dataset is robust to paradigm shifts. Federated learning degrades by only 0.40 percentage points on average, pointing to its suitability for multi-institutional collaboration scenarios where urban remote sensing data is distributed across agencies.
- **Salinas:** The largest dataset (54,129 samples) achieves the highest absolute semi-supervised accuracy of any configuration (3D-CNN + SSL: 97.30%) and the narrowest federated-supervised gap (−0.75 percentage points), confirming that both alternative paradigms scale favorably as labeled and unlabeled sample counts grow.

5.2.6 Privacy and Efficiency Trade-offs

Beyond raw accuracy, the three paradigms differ substantially in their operational characteristics. Table 5.14 summarizes these trade-offs qualitatively.

Table 5.14 Qualitative Comparison of Learning Paradigms

Metric	**Supervised**	**Semi-Supervised**	**Federated**
Data Privacy	Low	Low	High
Label Requirement	High	Medium	Low
Computational Cost	Low	Medium	High
Communication Overhead	None	None	Significant
Model Accuracy	Baseline	-15% avg (dataset-dependent)	-0.7% to -8%

6
CONCLUSIONS AND DISCUSSION

6.1 Summary

This project built and evaluated a unified HSI classification framework that places supervised, semi-supervised, and federated learning in direct comparison on the same preprocessing pipeline, the same three benchmark datasets (Indian Pines, Pavia University, and Salinas), and the same three architectures (2D-CNN, 3D-CNN, and Vision Transformer).

Raw spectral data exceeding 200 bands is conditioned through global min-max normalization, Gaussian spatial filtering, and PCA reduction to 30 components before being divided into $15 \times 15 \times 30$ patches. A stratified 90/10 train-test split preserves class proportions across both sets. The semi-supervised branch pairs self-training with curriculum learning, which schedules pseudo-labeled samples from high-confidence to ambiguous to dampen error propagation. The federated branch simulates a three-client environment where each dataset acts as a local data silo; local weights are aggregated at a central server via the FedAvg protocol without any raw data ever leaving the client.

6.2 Key Findings

6.2.1 Semi-Supervised Learning Effectiveness

Semi-supervised performance is strongly conditioned on dataset size:

1. On Pavia University and Salinas, where unlabeled pools are large, semi-supervised models achieve competitive accuracy (86–97%), confirming that self-training is a viable strategy given sufficient data volume.
2. On the smaller, class-imbalanced Indian Pines set, accuracy collapses to 54–74%—a clear signal that the confidence threshold and pseudo-label iteration count require dataset-specific tuning.

3. Across all datasets, 2D-CNN achieves the highest semi-supervised average at 89.22%, suggesting that simpler convolutional architectures are more forgiving of pseudo-label noise than deeper or attention-based ones.

4. Curriculum learning contains but does not eliminate the overfitting caused by noisy pseudo-labels; its benefit is most pronounced when the initial labeled set is large enough to seed reliable pseudo-labels.

6.2.2 Federated Learning Privacy-Performance Trade-off

Federated learning maintains near-supervised accuracy at the cost of communication overhead:

1. Federated 3D-CNN achieves a 97.36% average overall accuracy, representing only a -2.55 percentage point degradation relative to its centralized supervised counterpart (99.91%).

2. On Pavia University and Salinas, federated models reach 99–100% accuracy, demonstrating that FedAvg is robust when local data distributions are moderately heterogeneous.

3. The Indian Pines gap is wider (85–96% federated vs. 99–100% supervised), reflecting the difficulty of aggregating meaningful weight updates from a small local dataset across 50 communication rounds.

4. For organizations that cannot centralize raw spectral data, federated learning represents a pragmatically viable alternative—the accuracy cost is modest and the privacy guarantee is meaningful.

6.2.3 Architecture-Specific Insights

- **2D-CNN:** Leads overall accuracy in every paradigm (supervised: 99.96%, semi-supervised: 89.22%, federated: 98.72%). After PCA compresses the spectral axis to 30 components, 2D spatial convolutions capture sufficient discriminative information—volumetric operations add parameters without a proportional accuracy return.

- **3D-CNN:** Consistent second across all paradigms (supervised: 99.91%, semi-supervised: 86.92%, federated: 97.36%). Explicit modeling of inter-band correlations through volumetric kernels translates into more stable behavior under both noisy pseudo-labels and federated weight averaging.

- **Vision Transformer:** Competitive at full supervision (99.72%) but the most sensitive to distribution shift: semi-supervised average drops to 77.41% and federated to 94.11%. Global self-attention, while powerful under clean full-label conditions, is more easily destabilized by pseudo-label noise and client-side data heterogeneity.
- **Cross-paradigm pattern:** Every architecture deteriorates substantially under semi-supervised conditions relative to both supervised and federated baselines, which points to pseudo-label quality—not model capacity—as the binding constraint in the semi-supervised pipeline.

6.3 Project Objectives and Achievement Level

6.3.1 Original Objectives

The project initially set the following objectives:

1. Develop preprocessing pipelines handling high-dimensional hyperspectral data with robust normalization and dimensionality reduction.
2. Implement supervised learning baselines using multiple deep learning architectures (CNN, ViT).
3. Extend to semi-supervised learning via self-training and curriculum learning to address limited labeling.
4. Implement federated learning for privacy-preserving collaborative training across distributed data sources.
5. Develop a reactive GUI with multithreaded inference for practical accessibility.
6. Achieve $>$ 95% overall accuracy on benchmark datasets.

6.3.2 Achievement Level

All primary objectives were met or exceeded:

Table 6.1 Project Objectives and Achievement Level

Objective	Target	Achieved
Preprocessing Pipeline	Normalization + Filtering + PCA	✓ Fully Implemented
Supervised Baselines	Multiple architectures	✓ 2D-CNN, 3D-CNN, ViT
Semi-Supervised Learning	Self-training + CL	✓ Implemented (54–97% range)
Federated Learning	Multi-client aggregation	✓ FedAvg Protocol (85–99% range)
GUI with Multithreading	Python (PyQt6) + Multithreading	✓ Fully Operational
Target Accuracy (> 95%)	Supervised requirement	✓ Achieved (99.72–99.96%)

6.4 Strengths

6.4.1 Methodological Strengths

- **Unified comparative framework:** Evaluating three paradigms under identical preprocessing, split, and metric conditions is rare in the HSI literature; the results are directly comparable rather than confounded by differing experimental setups.
- **Curriculum Learning integration:** Coupling self-training with difficulty-ordered sample presentation measurably reduces pseudo-label overfitting and prevents early convergence to poor local minima.
- **Privacy-preserving federated architecture:** The federated system demonstrates that competitive classification accuracy is achievable without pooling sensitive spectral data, a result directly relevant to multi-institutional remote sensing collaborations.
- **Generalizable preprocessing:** The three-step pipeline (normalization, Gaussian filtering, fixed-component PCA) is data-driven and has been validated across three datasets with substantially different spatial resolutions, class counts, and band counts.

6.4.2 Implementation Strengths

- **Three-architecture support:** Users can select 2D-CNN, 3D-CNN, or ViT based on their accuracy-versus-efficiency requirements without modifying the surrounding pipeline.
- **GPU-accelerated training:** Google Colab GPU resources support efficient training on datasets exceeding 50,000 samples within realistic time budgets.

- **Responsive multithreaded GUI:** PyQt6 worker threads decouple inference from the UI event loop, keeping the interface interactive during long-running classification tasks.
- **Reproducibility:** Fixed random seeds and stratified splitting ensure that results are replicable across experimental runs without additional configuration.

6.5 Limitations and Weaknesses

6.5.1 Methodological Limitations

- **Simulated federated scenario:** Three local datasets on a single machine are a proxy for true distributed clients. Real deployments introduce network latency, stragglers, and far greater data heterogeneity—all of which would likely widen the accuracy gap.
- **Fixed PCA components:** Fixing $n\_components = 30$ is empirically motivated rather than theoretically grounded. Adaptive selection based on per-dataset variance retention or information-theoretic criteria could better match the complexity of each scene.
- **Confidence threshold rigidity:** The self-training confidence threshold of 0.95 is held constant across all datasets. Sensitivity analysis would reveal how robust the semi-supervised pipeline is to this choice, particularly on class-imbalanced sets.
- **Single curriculum schedule:** Only difficulty-based ordering is evaluated; reverse curriculum, paced learning, and uncertainty-driven schedules remain untested and may perform differently across dataset sizes.

6.5.2 Experimental Limitations

- **Benchmark datasets only:** Generalization to operational hyperspectral surveys, with real acquisition noise and non-standard class taxonomies, has not been tested.
- **No concept drift analysis:** The federated framework assumes stationary data distributions. Remote sensing data exhibits temporal drift that would require continual learning mechanisms in a production deployment.
- **Communication cost not quantified:** Federated accuracy results are reported, but bandwidth consumption and round-trip latency—critical for assessing real-world feasibility—are not measured.

- **Adversarial robustness not assessed:** No evaluation of model behavior under adversarial perturbations or sensor-induced distribution shift is conducted, both of which are relevant for operational deployment.

6.5.3 Implementation Limitations

- **No model compression:** Weight pruning or quantization would reduce federated communication overhead but is not implemented in the current system.
- **Single-GPU training:** Multi-GPU data parallelism is not supported, which limits scalability for datasets substantially larger than Salinas.
- **Narrow hyperparameter search:** Grid search covers a fixed candidate space; Bayesian optimization or AutoML approaches would likely identify better-tuned configurations, particularly for the semi-supervised confidence threshold.

6.6 Future Work

6.6.1 Methodological Extensions

1. **Continual federated learning:** Equip the federated system with mechanisms to adapt to temporal distribution shifts in hyperspectral data streams, enabling long-term deployment without full retraining.
2. **Heterogeneous federated aggregation:** Replace vanilla FedAvg with weighted aggregation schemes that account for statistical heterogeneity across clients (e.g., FedProx, SCAFFOLD).
3. **Adaptive curriculum generation:** Replace predefined difficulty orderings with automatic scheduling driven by model uncertainty or gradient-based difficulty estimation.
4. **Meta-learning for few-shot HSI:** Incorporate meta-learning to enable rapid adaptation to new sensors or geographic regions with only a handful of labeled samples.

6.6.2 Architectural Enhancements

1. **Hybrid 2D-3D models:** Combine 2D and 3D convolutional stages to balance spectral-spatial expressiveness against parameter count—the strong 2D-CNN results suggest this may be the most efficient frontier.

2. **Graph Neural Networks:** Model spectral-spatial relationships as graph structures, which may capture irregular spatial dependencies that regular convolutional grids miss.

3. **Self-supervised pretraining:** Apply contrastive or masked-reconstruction pretraining on unlabeled hyperspectral data to initialize richer representations before semi-supervised fine-tuning.

4. **Multi-modal fusion:** Extend the pipeline to fuse hyperspectral data with panchromatic imagery and LiDAR point clouds for scenes where a single modality is ambiguous.

6.6.3 Experimental Directions

1. **Real-world federated deployment:** Validate the federated system on actual data distributed across multiple institutions or space agencies (e.g., ESA, NASA) to confirm that simulated results transfer to production conditions.

2. **Cross-sensor generalization:** Benchmark federated learning across heterogeneous sensors (AVIRIS, Sentinel-2, DESIS) to characterize domain adaptation capability.

3. **Adversarial robustness:** Conduct systematic adversarial evaluation and develop sensor-noise-aware defense mechanisms for operationally deployed classifiers.

4. **Uncertainty quantification:** Integrate Bayesian deep learning to output per-prediction confidence estimates—essential for downstream decision-making in high-stakes remote sensing applications.

6.6.4 Practical Applications

1. **Edge deployment:** Compress models via pruning and quantization to enable real-time onboard classification on drones and nanosatellites.

2. **Active learning interface:** Add an active learning module to the GUI that identifies the most informative unlabeled pixels and routes them for human annotation, reducing labeling effort systematically.

3. **Interpretability tooling:** Embed explainable AI methods (attention maps, LIME, SHAP) so domain experts can interrogate model predictions before acting on them.

4. **GUI enhancements:** Extend the current interface with annotation support, quality assurance workflows, and real-time performance dashboards suited to operational remote sensing teams.

6.7 Conclusion

The experimental record is clear on two points. First, federated learning is a practical alternative to centralized training: a -3.1% average accuracy cost buys meaningful data privacy, and the gap narrows further on larger datasets. Second, semi-supervised learning via self-training and curriculum learning works well at scale—competitive on Pavia University and Salinas—but degrades substantially on the small, imbalanced Indian Pines set, where pseudo-label noise accumulates faster than the curriculum can contain it.

Among the three architectures, 2D-CNN is the most consistent performer across all paradigms, which is a practically important finding: after PCA reduces the spectral dimension, spatial convolution alone captures the discriminative structure needed for near-perfect classification. The Vision Transformer, despite matching CNN accuracy under full supervision, is the least robust to the distribution shifts introduced by pseudo-labeling and federated weight averaging.

The framework delivered here—a unified pipeline spanning three paradigms, three architectures, and three datasets, wrapped in a user-facing multithreaded GUI—provides both a reproducible comparative benchmark and a deployable application. Future work should prioritize real-world federated deployments and adaptive curriculum strategies that respond to the specific class-imbalance profile of each dataset, since those are the two factors that most constrain the system's current performance envelope.

References

[1] P. Du, Z. Xue, J. Li, and A. Plaza, "Learning discriminative sparse representations for hyperspectral image classification," *IEEE Journal of Selected Topics in Signal Processing*, vol. 9, no. 6, pp. 1089–1104, 2015.

[2] J. Xia, P. Ghamisi, N. Yokoya, and A. Iwasaki, "Random forest ensembles and extended multiextinction profiles for hyperspectral image classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 56, no. 1, pp. 202–216, 2017.

[3] Y. Zhang, G. Cao, X. Li, and B. Wang, "Cascaded random forest for hyperspectral image classification," *IEEE journal of selected topics in applied earth observations and remote sensing*, vol. 11, no. 4, pp. 1082–1094, 2018.

[4] C. Zhang, M. Han, and M. Xu, "Multi-feature classification of hyperspectral image via probabilistic svm and guided filter," in *2018 International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2018, pp. 1–7.

[5] X. Cao, X. Wang, D. Wang, J. Zhao, and L. Jiao, "Spectral–spatial hyperspectral image classification using cascaded markov random fields," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 12, no. 12, pp. 4861–4872, 2019.

[6] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE transactions on geoscience and remote sensing*, vol. 54, no. 10, pp. 6232–6251, 2016.

[7] W. Li, G. Wu, F. Zhang, and Q. Du, "Hyperspectral image classification using deep pixel-pair features," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 55, no. 2, pp. 844–853, 2016.

[8] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "Hybridsn: Exploring 3-d–2-d cnn feature hierarchy for hyperspectral image classification," *IEEE geoscience and remote sensing letters*, vol. 17, no. 2, pp. 277–281, 2019.

[9] M. Ahmad, A. M. Khan, M. Mazzara, S. Distefano, M. Ali, and M. S. Sarfraz, "A fast and compact 3-d cnn for hyperspectral image classification," *IEEE Geoscience and Remote Sensing Letters*, vol. 19, pp. 1–5, 2020.

[10] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "A new deep convolutional neural network for fast hyperspectral image classification," *ISPRS journal of photogrammetry and remote sensing*, vol. 145, pp. 120–147, 2018.

[11] S. Li, X. Zhu, Y. Liu, and J. Bao, "Adaptive spatial-spectral feature learning for hyperspectral image classification," *IEEE access*, vol. 7, pp. 61 534–61 547, 2019.

[12] J. Zhao, L. Hu, Y. Dong, L. Huang, S. Weng, and D. Zhang, "A combination method of stacked autoencoder and 3d deep residual network for hyperspectral image classification," *International Journal of Applied Earth Observation and Geoinformation*, vol. 102, p. 102 459, 2021.

[13] C. Shi and C.-M. Pun, "Multi-scale hierarchical recurrent neural networks for hyperspectral image classification," *Neurocomputing*, vol. 294, pp. 82–93, 2018.

[14] L. Zhu, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Generative adversarial networks for hyperspectral image classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 56, no. 9, pp. 5046–5063, 2018.

[15] X. He, Y. Chen, and Z. Lin, "Spatial-spectral transformer for hyperspectral image classification," *Remote Sensing*, vol. 13, no. 3, p. 498, 2021.

[16] P. Zhang, H. Yu, P. Li, and R. Wang, "Transhsi: A hybrid cnn-transformer method for disjoint sample-based hyperspectral image classification," *Remote Sensing*, vol. 15, no. 22, p. 5331, 2023.

[17] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral–spatial feature tokenization transformer for hyperspectral image classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–14, 2022.

[18] Z. Shu, Y. Wang, and Z. Yu, "Dual attention transformer network for hyperspectral image classification," *Engineering Applications of Artificial Intelligence*, vol. 127, p. 107 351, 2024.

[19] A. Sellami, M. Farah, I. R. Farah, and B. Solaiman, "Hyperspectral imagery classification based on semi-supervised 3-d deep neural network and adaptive band selection," *Expert Systems with Applications*, vol. 129, pp. 246–259, 2019.

[20] M. F. Baumgardner, L. L. Biehl, and D. A. Landgrebe, "220 band aviris hyperspectral image data set: June 12, 1992 indian pine test site 3," *Purdue University Research Repository*, 2015.

[21] P. Dell'Acqua, P. Gamba, et al., "A comparison of svm and rbf neural networks for the classification of hyperspectral images," *IEEE Transactions on Geoscience and Remote Sensing*, 2005.

[22] D. A. Landgrebe et al., *Salinas hyperspectral image dataset*, 1998.

[23] F. Pedregosa et al., "Scikit-learn: Machine learning in python," *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.

[24] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021.

[25] I. Triguero, S. García, and F. Herrera, "Self-labeled techniques for semi-supervised learning: Taxonomy, software and empirical study," *Knowledge and Information Systems*, vol. 42, no. 2, pp. 245–284, 2015.

[26] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th annual international conference on machine learning*, 2009, pp. 41–48.

[27] G. Van Rossum and F. L. Drake Jr, *Python tutorial*. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995.

[28] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.

[29] M. Abadi et al., "Tensorflow: A system for large-scale machine learning," in *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, 2016, pp. 265–283.

[30] Itseez, *Open source computer vision library*, `https://github.com/itseez/opencv`, 2015.

[31] Microsoft, *Visual studio code*, `https://code.visualstudio.com/`, 2015.

[32] L. Dabbish, C. Stuart, J. Tsay, and J. Herbsleb, "Social coding in github: Transparency and collaboration in an open software repository," in *Proceedings of the ACM 2012 conference on computer supported cooperative work*, 2012, pp. 1277–1286.

[33] E. Bisong, "Google colaboratory," in *Building Machine Learning and Deep Learning Models on Google Cloud Platform: A Comprehensive Guide for Beginners*, Springer, 2019, pp. 59–64.

[34] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*, PMLR, 2017, pp. 1273–1282.

Curriculum Vitae

FIRST MEMBER

Name-Surname: TAN ERCİYAS
Birthdate and Place of Birth: 15.12.2003, Trabzon
E-mail: tan.erciyas@std.yildiz.edu.tr
Phone: 0505 353 60 58
Practical Training: Genarion Software Department
Baykar Web Software Department
Tüpraş Software Department

SECOND MEMBER

Name-Surname: KEREM BAYDAR
Birthdate and Place of Birth: 06.11.2004, Kocaeli
E-mail: kerem.baydar@std.yildiz.tr
Phone: 0544 735 41 00
Practical Training: Baykar OOP Software Department
IBTECH Dw & Data Insights Department
Genarion Software Department
Uzer Makina IT Department

Project System Informations

System and Software: Windows İşletim Sistemi, Python
Required RAM: 2GB
Required Disk: 256MB